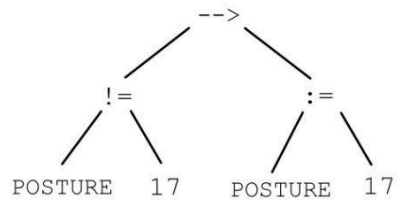
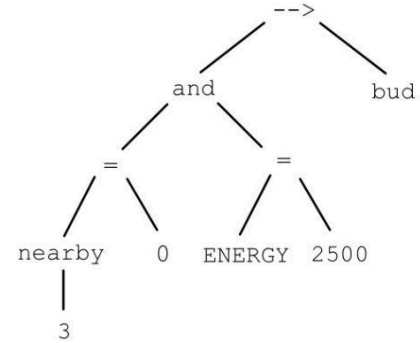


1. Drawing AST

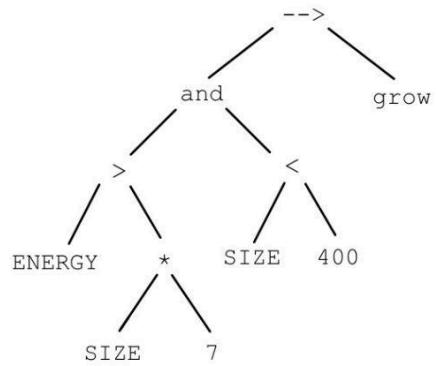
Rule 1



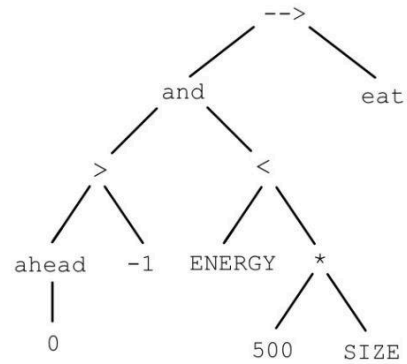
Rule 2



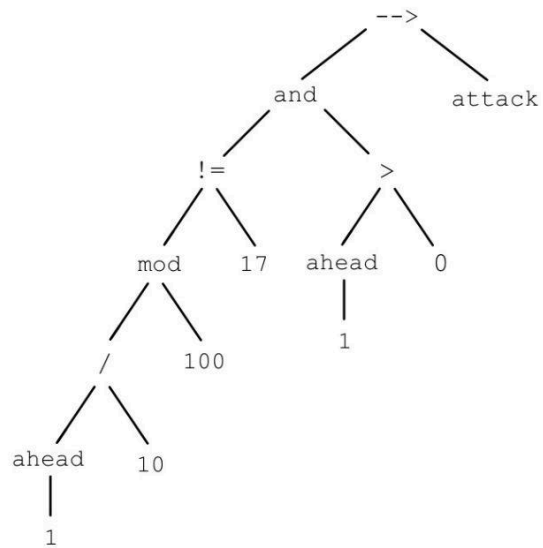
Rule 3



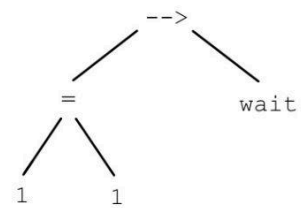
Rule 4



Rule 5



Rule 6



2. Loop Invariant

Invariant: **min** contains the minimum value and **max** contains the maximum value in the subarray **a[0]** to **a[i-1]** and **i** is even.

Establishment:

Since **i = 0**, **i** is even and there are no elements in the subarray **a[0]** to **a[i-1]**, thus, the invariant holds vacuously. The starting value of **min = Integer.MAX_VALUE** and **max = Integer.MIN_VALUE** only serves to ensure any value in the array will update them correctly and doesn't affect the loop invariant at this moment.

Preservation:

Let i_0 be the **i** value at the start of the loop and i_f be the **i** value at the end of the loop. In each loop iteration we will look at the values at **a[i₀]** and **a[i₀+1]**. Then, the if statement on line 7 initializes **x** to the smaller value and **y** to the larger value of the two. Then, we compare **min** and **x** and set **min** to the smaller value of the two through **min = Math.min(min, x)**, we also compare **max** and **y** and set **y** to the larger value of the two through **max = Math.max(max, y)**. At the end of the loop iteration, **i** is incremented by 2, that is, $i_0 + 2 = i_f$, the part of the invariant that states **i** must be even is satisfied since additions of even numbers will always result in an even number. Given that **min** and **max** are the minimum and maximum of the subarray **a[0]** to **a[i₀-1]** and now holds the minimum and maximum of the subarray **a[0]** to **a[i₀+1]**, **min** and **max** now must contain the minimum and maximum values in the subarray **a[0]** to **a[i_f-1]**, thus our invariant must hold by the end of the loop.

Therefore whenever both the loop invariant and the guard are true just before executing the loop body, then the loop invariant must be true just after executing the loop body.

Post-Condition:

Given that the precondition states that the array length must be even, **i** is always even, and the guard is **(i < a.length-1)**, **i** must be equal to **a.length** when the guard is false. This is because **a.length - 1** must be an odd number since **a.length** is even, and so **i** must be the smallest even number that is not smaller than **a.length - 1**, which is **a.length**. If the loop invariant holds, then **min** contains the minimum value and **max** contains the maximum value in the subarray **a[0]** to **a[i-1]**. And since **i** is equal to **a.length** then **a[i-1]** is the last element in the array, and so **min** and **max** contain the minimum and maximum values in the array, as required.

3. Loop Invariant for Polynomial

Invariant: all elements in the subarray `coeffs[n-1:]` (from `coeffs[n-1]` to the end of the array) are equal to zero.

Establishment:

When the loop begins `n = coeffs.length`, meaning there are no trailing elements in the array, and our loop invariant holds.

Postcondition:

If `(n > 0 && coeffs[n - 1] == 0)` is false and the invariant is true, then either:

- `n <= 0` which means all the values in the array are equal to zero.
- `coeffs[n - 1] != 0` which means the value at `coeffs[n - 1]` is not zero.

In both these cases, the desired state where `n` equals the number of elements at the beginning of the array without trailing zeros.

Preservation:

Each time the loop guard `(n > 0 && coeffs[n - 1] == 0)` is true, it implies that the value at `coeffs[n - 1] == 0` and then decrements `n` by one. Therefore ensuring any values coming after element `coeffs[n - 1]` are all zero.

Termination:

In each iteration `n` decreases by 1 until it is less than or equal to zero, at which point the guard statement is false. Thus, the loop must terminate.

Therefore, by establishing a loop invariant and proving the establishment, postcondition, preservation and termination of the loop, this loop is correct.